

Library Content & Course-Authoring Permissions

How the course library is laid out — courses, modules, lessons, materials and taxonomy — and exactly who is allowed to create, upload and edit each part. Extracted from the codebase: application-layer checks and PostgreSQL Row-Level Security.

Content hierarchy & visibility

Role & ownership model

Storage buckets & upload paths

RLS policy catalog

Scope: Course/library content structure & write-access model · **Repository:** shashankraghram1509/learnify-spotify

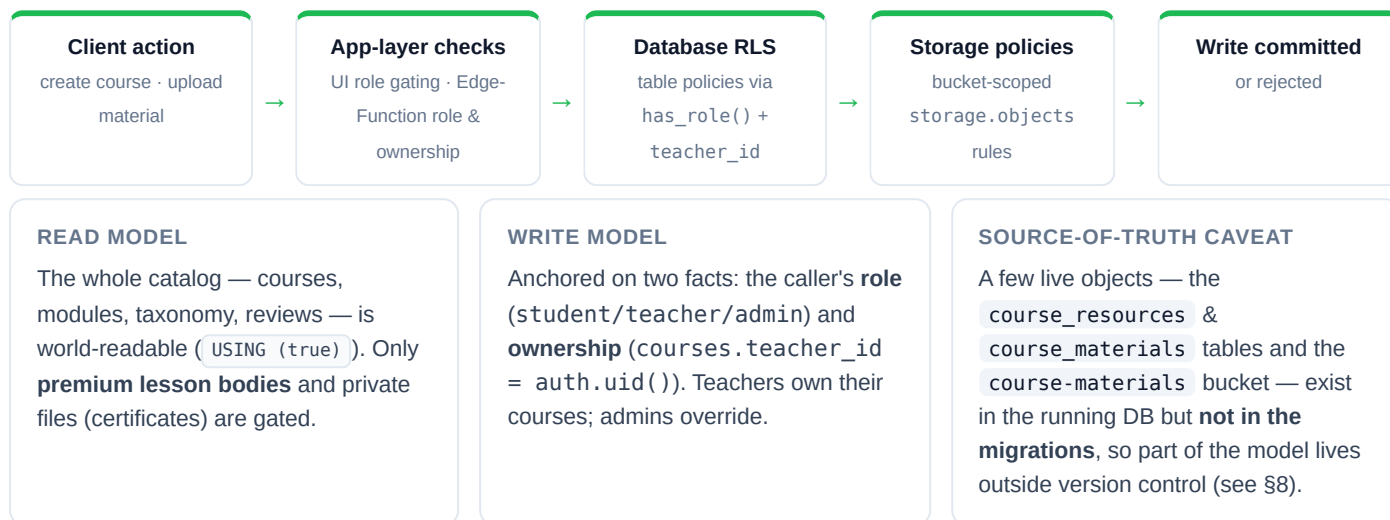
Sources: 36 SQL migrations · Storage policies · authoring components · upload-to-drive function · **Date:** 28 July 2026

01 Overview — a two-layer permission model

Learnity — Library Content & Course-Authoring Permissions · Architecture

From source at commit 72b453b · 2026-07-28

The library's content is **public to read** but tightly governed to write. Every write is checked in up to two places: an **application layer** (UI gating and, for the Drive path, explicit role + ownership checks inside an Edge Function) and the **database layer** — PostgreSQL **Row-Level Security (RLS)** policies on the content tables plus policies on the Storage buckets. RLS is the real backstop: even if the UI is bypassed, the database rejects an unauthorized write.



02 How the library is laid out

The content hierarchy that makes up a course, with each level's read-visibility. **PUBLIC** world-readable · **GATED** conditional · **PRIVATE** owner-only.



The catalog is the library. Because `courses`, `modules`, taxonomy and resources are all world-readable, browsing/search works for anyone. The single access boundary inside a course is the **premium-lesson gate**: a premium course's lesson bodies are visible only to enrolled students, enforced directly in the `lessons` RLS policy — not just hidden in the UI.

03 Roles & how they are granted

Learnity — Library Content & Course-Authoring Permissions - Architecture

From source at commit 72b453b · 2026-07-28

Authoring rights flow entirely from a user's role. Roles live in their own table and are deliberately hard to self-assign.

THE ROLE MODEL

- Roles are `app_role ∈ {student, teacher, admin}`, stored in a dedicated `user_roles` table — **separated from profiles for security**
- Checked via `has_role(user, role)` — a **SECURITY DEFINER** SQL function that reads `user_roles`, so RLS policies can call it without recursion
- A user may hold multiple role rows (`UNIQUE(user_id, role)`)

HOW YOU GET A ROLE

- **Signup → always student.** The `handle_new_user` trigger forces the student role and **ignores any client-supplied role** in metadata (locked down after earlier versions trusted it)
- **Teacher / admin are provisioned by an admin** — an RLS policy lets admins insert teacher role rows; the `admin-create-instructor` function does this server-side
- Net effect: elevated authoring rights can't be self-granted

Design intent, verbatim from the migration: “Always assign ‘student’ role by default, ignoring any client input. Teachers and admins must be created through secure admin interface.” This is the linchpin of the authoring model — content-write power is bootstrapped only by an existing admin.

04 Who can author what

Effective write permissions across the library. ● allowed · ▲ conditional · ● denied. "Owner" = the course's `teacher_id`.

ACTION	STUDENT	TEACHER · OWNER	TEACHER · OTHER	ADMIN	NOTES
View catalog & content	●	●	●	●	premium lesson bodies need enrollment
Create a course	●	●	●	●	teacher role only — not bound to own id (§8)
Edit / delete <i>own</i> course	●	●	●	●	owner + teacher role; admin = any
Edit / delete <i>any</i> course	●	●	●	●	“Admins can manage all courses”
Manage modules & lessons	●	●	●	▲ no policy	owner-only; no admin RLS policy exists (§8)
Upload file → <i>course-materials</i>	●	▲ own path	●	●	path must start with owned course id
Add <i>course_resources</i> row	●	●	●	▲	Drive fn: teacher role + ownership
Upload thumbnail / video	●	●	●	●	admin-only buckets
Manage categories / tags	●	●	●	●	no write policy → service-role only
Grant teacher / admin role	●	●	●	●	admins insert teacher rows

05 Course creation & editing flows

Learnity — Library Content & Course-Authoring Permissions - Architecture

From source at commit 72b453b · 2026-07-28

Two authoring surfaces write to the same `courses` table — one for teachers (self-service) and one for admins (full control).

Teacher — self-service

teacher/MyCourses.tsx

- “Create Course” dialog → direct client `insert` into `courses` with `teacher_id = self`
- Manages materials inline per course (upload + list)
- Delete via client `delete` ; edit button present but not yet wired
- Governed by RLS: `INSERT: has_role(uid, 'teacher')` and `UPDATE/DELETE: uid = teacher_id AND has_role(uid, 'teacher')`

Admin — full management

admin/CourseManagement.tsx

- Sees **every** course; can create, edit & delete any of them
- **Assigns a course to any teacher** (sets `teacher_id` from a picker)
- Uploads cover & video files to Storage (`course-thumbnails` , `course-videos`)
- Governed by RLS: `ALL: EXISTS(user_roles WHERE uid AND role='admin')`

Modules & lessons are written only through the owning teacher's context — the RLS policy is

`ALL USING EXISTS(courses WHERE id = *.course_id AND teacher_id = auth.uid())` . There is **no admin policy on modules or**

lessons, so an admin editing another teacher's course cannot alter its module/lesson structure through the normal client (only `courses` - level fields).

06 Material & file upload architecture

Four buckets, and two distinct code paths for adding course materials — enforcing permission in different places.

PATH A — SUPABASE STORAGE (IN USE)

`MaterialsUpload.tsx` uploads the file to bucket `course-materials` at path `{courseId}/{timestamp}.{ext}`, gets a public URL, then client- `insert` s a `course_resources` row. **Permission rests on the Storage policy** — the path's first segment must be a course the caller owns.

PATH B — GOOGLE DRIVE (HARDENED)

`upload-to-drive` Edge Function does **explicit checks in code** — caller's `user_roles.role === 'teacher'` and `courses.teacher_id === caller` — then uploads to Google Drive and inserts the `course_resources` row pointing at the Drive link.

BUCKET	VISIBILITY	LIMITS	WHO CAN WRITE	PATH RULE
course-materials	referenced	none in migrations	Owning teacher (INSERT only)	1st segment = owned course id
course-thumbnails	public	5 MB · images	Admins (ins/upd/del)	none
course-videos	public	500 MB · video	Admins (ins/upd/del)	none
certificates	private	—	Owner (per-user folder)	1st segment = own user id

Enforcement lives in different places. Materials (teacher) are guarded by a **path-scoped Storage policy**; thumbnails/videos (admin) by **role-only Storage policies with no path restriction**; the Drive path by **explicit code checks**. The `course-materials` bucket and the `course_resources` table are not created by any migration, so their full rule-set lives in the running database rather than version control.

07 RLS policy catalog

Learnity — Library Content & Course-Authoring Permissions - Architecture

From source at commit 72b453b · 2026-07-28

The exact policies governing content, as found in the migrations. SELECT INSERT UPDATE DELETE ALL

Content tables

TABLE	CMD	PREDICATE (USING / WITH CHECK)
courses	SELECT	true — anyone can view
	INSERT	has_role(auth.uid(), 'teacher')
	UPD DEL	auth.uid() = teacher_id AND has_role(auth.uid(), 'teacher')
	ALL	EXISTS(user_roles WHERE user_id = auth.uid() AND role = 'admin') — admin override
modules	SELECT	true
	ALL	EXISTS(courses WHERE id = modules.course_id AND teacher_id = auth.uid())
lessons	SELECT	free course (c.is_premium = false OR NULL) — anyone
	SELECT	premium course AND enrolled (enrollments.student_id = auth.uid())
	ALL	EXISTS(modules→courses WHERE c.teacher_id = auth.uid())
reviews	SELECT	true
	INSERT	auth.uid() = user_id AND EXISTS(enrollments for this course & student)
	UPD	auth.uid() = user_id
categories, tags, course_categories, course_tags	SELECT	true — no INSERT/UPDATE/DELETE policy → client writes denied (service-role only)
course_resources course_materials	—	Not in migrations — no CREATE TABLE / RLS in version control; governed by live-DB RLS + the Drive function's checks

Storage · storage.objects policies

BUCKET	CMD	PREDICATE
course-materials	INSERT	bucket_id='course-materials' AND EXISTS(courses WHERE id::text = foldername[1] AND teacher_id = auth.uid())
course-thumbnails	SELECT	bucket_id='course-thumbnails' — public
	INS UPD DEL	EXISTS(user_roles WHERE user_id = auth.uid() AND role = 'admin')
course-videos	SELECT	bucket_id='course-videos' — public
	INS UPD DEL	EXISTS(user_roles WHERE user_id = auth.uid() AND role = 'admin')
certificates	SELECT	bucket_id='certificates' AND auth.uid()::text = foldername[1]
	INSERT	bucket_id='certificates' AND auth.uid()::text = foldername[1]

Enrollment & certificates, for context. Direct client writes are **blocked** on both: enrollments insert only via `service_role` (the `create-enrollment` function), and certificates insert only via a `SECURITY DEFINER` completion trigger — so authoring privileges never leak into records that must be system-generated.

08 Findings, gaps & recommendations

Learnity — Library Content & Course-Authoring Permissions · Architecture

From source at commit 72b453b · 2026-07-28

Observations that matter for correctness and security of the authoring model, most notable first.

1 · Course creation isn't bound to the creator. The `courses` INSERT policy checks only `has_role(auth.uid(), 'teacher')` — it never requires `teacher_id = auth.uid()`. Any teacher can create a course attributed to *another* teacher's id (and would then be unable to edit it, since UPDATE/DELETE require ownership). **Fix:** add `WITH CHECK (teacher_id = auth.uid())` to the INSERT policy.

2 · Part of the model lives outside migrations. `course_resources`, `course_materials` and the `course-materials` bucket exist in the running database but have **no CREATE/RLS statements in any migration**. Their access rules can't be reviewed, diffed, or reproduced from source. **Fix:** capture them as migrations so the whole permission surface is in version control.

3 · ADMINS CAN'T EDIT MODULES/LESSONS

"Admins can manage all courses" covers only the `courses` table. `modules` and `lessons` have **owner-teacher-only** policies and no admin clause, so an admin cannot restructure another teacher's curriculum via the client. Add admin clauses if that's intended.

4 · TWO UPLOAD PATHS, UNEVEN GUARANTEES

The Storage path (`MaterialsUpload`) has no in-code ownership check and leans entirely on the bucket policy; the Drive path checks explicitly. Keeping both means two rule-sets to maintain. Consider standardising on one.

5 · ADMIN BUCKETS HAVE NO PATH SCOPING

Thumbnail/video policies are role-only — any admin may write anywhere in those buckets. Low risk (trusted role), but a per-course path convention would tighten it.

6 · ROLE LOOKUP USES `.SINGLE()`

Because `user_roles` allows multiple rows per user, the Drive function's `.select('role')...single()` would error for a user holding more than one role. Prefer `has_role()` or a `.in()` check.

Bottom line. The library is a **public catalog with a single premium-lesson gate**, and authoring is a clean **role + ownership** model: students can't write, teachers own their courses and materials, admins override at the course level, and system-only records (enrollments, certificates) are sealed off from direct writes. The main hardening opportunities are binding course creation to its owner, and bringing the out-of-migration objects back under version control.